CloudPulse: Closing the Loop from Cloud Cost Signal to Verified Saving with Read-Only Azure Access, Transparent Statistics and Owner Attribution

Tamil Mani B, Theja J G, Varunesh D, Viswajith R

Department of Computer Science and Engineering, R.M.K. College of Engineering and Technology (An Autonomous Institution), Puduvoyal, Tamil Nadu 601 206, India

Corresponding author e-mail: [add corresponding author e-mail]

Abstract. Cloud spending is easy to start and hard to control. Forgotten virtual machines, orphaned disks and snapshots, idle public addresses and unexplained cost spikes accumulate quietly, and most cost tools stop at a dashboard that reports what was spent without telling anyone who should act or whether the action helped. This paper presents CloudPulse, an open cloud cost monitoring and optimisation platform that carries each cost problem through one complete loop: signal, context, finding, owner, action and verification. CloudPulse reads a Microsoft Azure subscription through a service principal that holds only the Reader and Cost Management Reader roles, so it can observe but never change the environment. Detection uses transparent methods whose every decision can be inspected: a weekday-aware robust z-score (median and median absolute deviation) for spend anomalies, Holt's linear smoothing with weekday factors for month-end and budget forecasts, percentile verdicts for idle and oversized machines, and rule checks for unattached disks, idle addresses and stale snapshots. Findings are priced, scored from P1 to P4, attributed to the identity that created the resource using the Azure Activity Log before tags, and counted as savings only after seven days of billed cost confirm the drop. We describe the design, give the algorithms in full, and evaluate them on the FinOps Foundation's public FOCUS sample bill, with 27 automated tests, and with a seeded simulation. In the simulation the detector found 96–100% of injected spikes of twice the usual cost and matched or exceeded a mean-plus-three-sigma rule at every spike size tested, at the price of more false alarms, and Holt forecasting with weekday factors reduced seven-day-ahead error against a seven-day moving average (9.99% against 11.51% MAPE). We state the limits of this evaluation plainly: it is a functional and simulated evaluation, not a field study.

Keywords: FinOps; cloud cost optimisation; anomaly detection; robust statistics; Holt forecasting; Microsoft Azure; least privilege; FOCUS

I. Introduction

Cloud platforms bill by the second or the hour, and every resource created from a portal, a command line or an automation script starts accumulating cost the moment it exists [1]. The same elasticity that makes the cloud attractive makes waste easy: resources are quick to create and easy to forget. A virtual machine stopped from inside its operating system is reported as stopped yet still reserves its compute allocation; a disk survives the machine it served; a snapshot outlives the disk it protected; a public IP address stays billed after the service behind it is gone. Practitioner literature on cloud financial management treats this as a central problem of the discipline now called FinOps [2].

Existing tools address part of it. Provider consoles show the bill and issue recommendations; open-source policy engines such as Cloud Custodian [3] act on rules; commercial platforms add reporting. In our experience as students building and running small Azure workloads, three gaps remained. First, a recommendation names a resource but not the person who created it, so nobody is accountable. Second, tools that report a saving rarely check afterwards that the bill actually fell. Third, a tool that is allowed to delete resources needs credentials that are dangerous if leaked, while a read-only tool is safe but usually stops short of explaining its findings.

CloudPulse is our attempt to close these gaps with deliberately simple, inspectable methods. This paper makes four contributions:

- **A read-only collection design.** CloudPulse authenticates as an Entra ID service principal with Reader and Cost Management Reader only, gathers cost, inventory, metrics and audit data through a small REST client, and stores secrets encrypted; it has no write path to the customer's cloud.
- **Transparent detection.** Each detector is a short, documented rule or statistic, and each finding carries the evidence that triggered it.
- **Ownership and a finding lifecycle.** Findings are attributed to the creating identity from the Azure Activity Log, with tags second, and move through a defined lifecycle.
- **Verified savings.** A saving is counted only after a seven-day before-and-after comparison of the billed cost, and a fixed resource that returns reopens its finding.

The remainder of the paper reviews related work (Section II), describes the architecture (III) and the algorithms (IV), reports the implementation (V) and evaluation (VI), and concludes with limitations and future work (VII).

II. Background and Related Work

Cloud cost management. Armbrust et al. identified pay-per-use pricing and elasticity as defining properties of cloud computing [1]. The FinOps community has since formalised practices for allocating, optimising and governing cloud spend [2], and published the FOCUS specification, a vendor-neutral schema in which Amazon Web Services, Microsoft Azure, Google Cloud and Oracle can export billing data [4]. CloudPulse imports FOCUS files, which lets its cost analytics run on any provider's bill, while its resource-level checks currently target Azure.

Anomaly detection in cost series. The classical rule of flagging values more than three standard deviations from the mean is fragile, because the outliers it should find inflate the mean and the standard deviation it relies on. Leys et al. recommend the median and the median absolute deviation (MAD) instead [5], and Rousseeuw and Croux discuss scaling the MAD by 1.4826 to estimate the standard deviation of normal data [6]. Iglewicz and Hoaglin propose a modified z-score with a conventional cut-off of 3.5 [7]. CloudPulse uses this robust score, but computes it against the same weekday of earlier weeks, because cloud spend has a strong weekly pattern.

Forecasting. Holt's linear exponential smoothing tracks a level and a trend [8], and is covered with its seasonal variants in standard texts [9]. It needs little data and no training pipeline, which suits a workspace that has only a few weeks of history.

Security of delegated access. Least privilege, the principle that an identity should hold only the permissions its task needs, is a long-standing design rule [10]. Azure applies it through role-based access control, where a security principal, a role and a scope together form an assignment [11], and a service principal obtains tokens through the OAuth 2.0 client-credentials grant [12]. CloudPulse stores cloud secrets with Fernet authenticated encryption [13] and user passwords with the scrypt memory-hard function [14].

III. System Architecture

Figure 1 shows the three tiers. Browser users (administrator, customer owner, customer viewer and an anonymous demo visitor) reach a web interface served by a FastAPI application. The application holds the API and security layer, a scheduler, a collection orchestrator, the collectors, the detectors,

the findings engine, an owner resolver and a price book, all above a data-access layer. State lives in an SQL database (SQLite for development, PostgreSQL for deployment) beside a credential store. The right-hand side is the customer's Azure subscription.

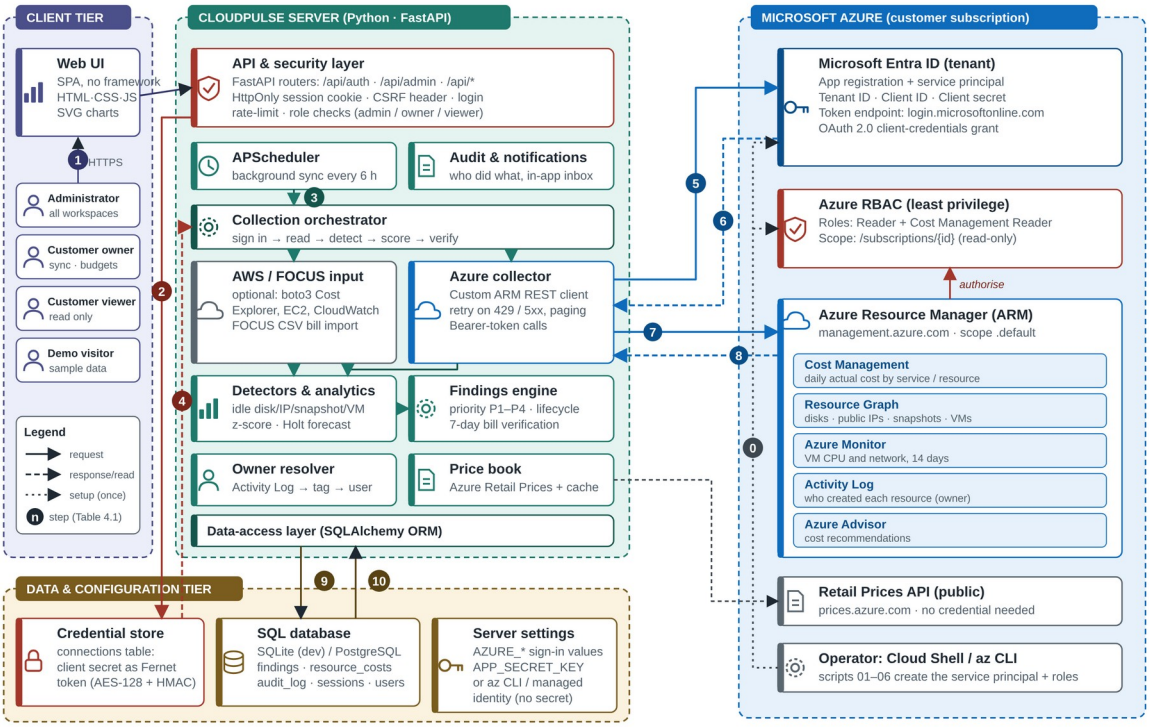


Figure 1 - CloudPulse architecture with the Azure credential and data flows

Credential flow. An operator runs a set of Azure CLI scripts once. They create an app registration and service principal and assign the two read-only roles at subscription scope (step 0 in Figure 1). On each synchronisation the collector presents the tenant identifier, client identifier and client secret to Microsoft Entra ID, receives a bearer token (steps 5 and 6), and calls Azure Resource Manager (steps 7 and 8). Azure RBAC authorises every call against the assigned roles, so a stolen secret could read cost and inventory but could not create, modify or delete anything. Three sign-in modes are supported: the developer's own Azure CLI session on a laptop, a service principal anywhere, and a managed identity on Azure App Service, in which case no secret exists at all.

Data sources. The collector reads daily cost by service and resource from Cost Management; disks, public IP addresses, snapshots and virtual machines (including power state) from Resource Graph; hourly CPU and network metrics for fourteen days from Azure Monitor; the creator of each resource from the Activity Log; and recommendations from Azure Advisor. A custom REST client retries on throttling (HTTP 429) and server errors and follows result paging. Prices for idle resources come from the public Azure Retail Prices API, with a local cache.

IV. Methods

A. Spend anomaly detection

Let x_t be the cost on day t. The baseline B_t is the same weekday in the previous four weeks when at least three of those days have billing data, and the previous fourteen days otherwise. With m_t the median of B_t and MAD_t its median absolute deviation, the spread and the robust score are

$$s_t = \max(1.4826 \cdot MAD_t,\ 0.05 \cdot m_t,\ 0.01), \quad z_t = (x_t - m_t) / s_t \qquad (1)$$

A day is flagged when z_t > 3.5 and the excess x_t − m_t is at least max(1, 0.2 · m_t). The floors on s_t stop a perfectly flat history from producing a zero spread, and the excess condition suppresses statistically significant but financially trivial spikes. At least seven billed days of history are required. The cause of a flagged day is explained by comparing each service's cost that day with its own median on the baseline days; the positive differences are normalised to shares that add to 100%. A spike is classed as ongoing when the following two days both exceed m_t + 0.5 (x_t − m_t); an ongoing spike is projected at 30.4 times the daily excess per month, and a one-off spike is reported at its own excess. Confidence is min(0.95, 0.55 + 0.05 (z_t − 3.5)).

B. Month-end and budget forecasting

Daily cost is first divided by a weekday factor, then smoothed with Holt's method:

$$\ell_t = \alpha\, y_t + (1-\alpha)(\ell_{t-1} + b_{t-1}), \quad b_t = \beta\,(\ell_t - \ell_{t-1}) + (1-\beta)\, b_{t-1} \qquad (2)$$

The pair (α, β) is chosen from the grid α ∈ {0.2, 0.4, 0.6, 0.8}, β ∈ {0.05, 0.1, 0.2} by minimising the one-step-ahead squared error. The h-day forecast is ℓ_t + h b_t, multiplied back by the weekday factor, and an 80% interval uses 1.2816 times the residual standard deviation. The projected month-end total is compared with the workspace budget, and a budget-risk finding is raised when the projection exceeds it; the predicted crossing day is shown on the chart.

C. Idle and oversized machines

From at least 48 hourly Azure Monitor points, CloudPulse computes the 95th percentile of hourly CPU and of network traffic. A machine is idle when its CPU p95 is below 5% and network p95 below 5 MB per hour, and oversized when CPU p95 is below 20% and the peak is below 80%. Confidence is 0.85 with seven or more days of data and 0.65 with less; oversized verdicts are reduced by 0.15 because memory is not measured. An oversized finding proposes the next smaller size from the same series. A percentile rather than the mean prevents a brief batch job from hiding an otherwise idle machine.

D. Rule checks

Four checks use resource state directly: a managed disk whose state is Unattached; a static public IP address with no association; a snapshot whose source disk no longer exists or that is older than a configurable age; and a virtual machine that is stopped but not deallocated, whose impact is its compute price plus the price of its disks. Each check records the rule that fired as evidence, and each proposes an Azure CLI command, with a backup step where deletion is irreversible.

E. Priority score

Each finding receives a score between 0 and 100:

$$S = 100 \cdot (0.6 \cdot I + 0.4 \cdot W) \cdot C \cdot A \qquad (3)$$

where I = min(1, log10(1 + impact) / log10(1 + 1000)) is a logarithmic impact term (impact in currency units per month; governance findings such as untagged spend are weighted down), W is a severity weight (critical 1.0, high 0.8, medium 0.55, low 0.3), C is the confidence, and A = 1 + min(0.3, days_open / 100) lets neglected findings rise. Scores of at least 70, 45 and 25 map to P1, P2 and P3, and the rest to P4. The logarithm stops a single very large item from drowning everything else (Figure 2).

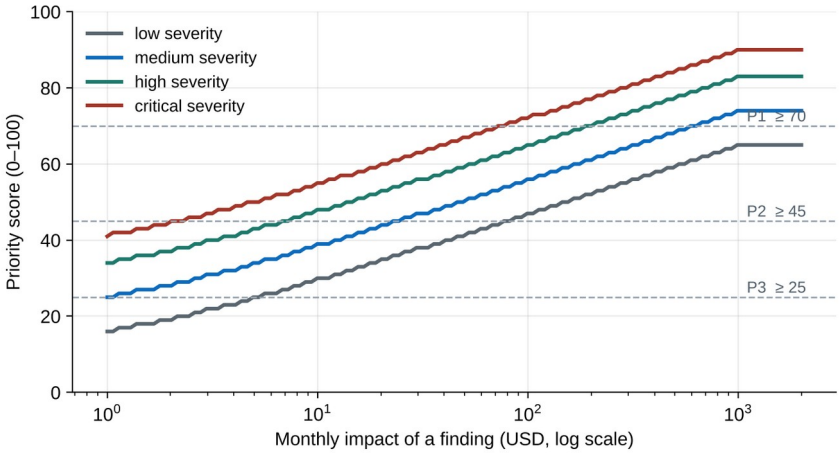


Figure 2 - Priority score against monthly impact (confidence 0.9, new finding), produced by the implemented function

F. Ownership and lifecycle

The owner of a resource is resolved in order from the Azure Activity Log (the identity that created it), then from an owner tag, then from a manual assignment; when none exists the finding is shown as having no owner rather than guessed. An owner is matched to a CloudPulse user where the e-mail address agrees. Findings move through the states of Figure 3: open, assigned, in progress, resolved and verified, with dismissal (a reason is required) and snoozing from any active state. A resolved finding returns to open if the resource is detected again, and a verified saving is withdrawn.

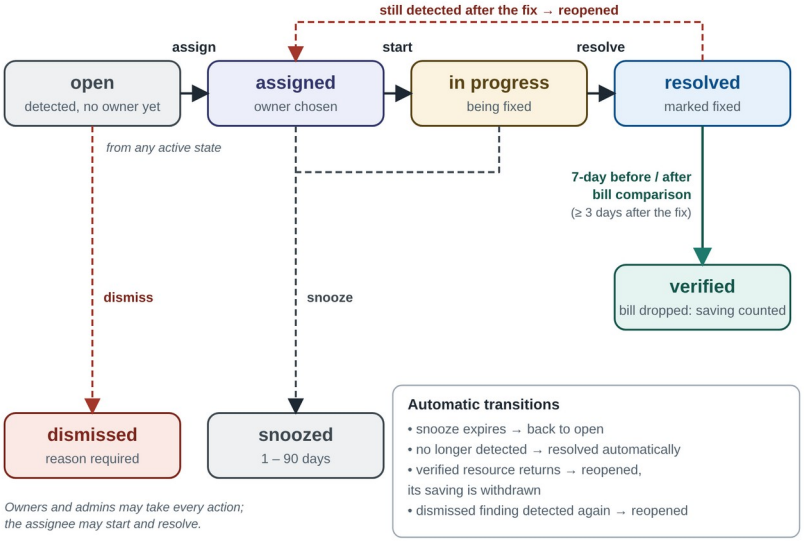


Figure 3 - Finding lifecycle, with automatic and manual transitions

G. Verification of savings

When a finding is marked fixed on day d, CloudPulse compares the average daily cost of the affected resource over the seven days before d with the days after d (excluding d, which may be billed partly). Missing cost days count as zero only up to the last day for which the workspace has any billing data, because Azure publishes cost with a delay of up to a day. No result is produced until at least three days after the fix are available. The realised monthly saving is the daily drop multiplied by 30.4, and the ratio of realised to estimated saving gives an estimate-accuracy figure for the workspace.

V. Implementation

The back end is written in Python with FastAPI and SQLAlchemy, with APScheduler running a synchronisation every six hours. The interface is plain HTML, CSS and JavaScript with hand-written SVG charts and no front-end framework, which keeps the whole product small enough to read. Sessions use an HttpOnly cookie with a CSRF header and login rate limiting; three roles (administrator, customer owner, customer viewer) and a read-only demo identity control access; every sensitive action is written to an audit log. Stored cloud secrets are encrypted with Fernet; passwords are hashed with scrypt. A FOCUS importer aggregates line items to one row per provider, service and day, keeping the share of cost that carried tags so that untagged spend can be reported. Table 1 lists the components.

Table 1 - Main components and technologies

Component	Technology	Purpose
API and server	Python, FastAPI, Uvicorn	REST endpoints, authentication, role checks
Storage	SQLAlchemy; SQLite or PostgreSQL	Costs, findings, users, audit log, encrypted credentials
Scheduler	APScheduler	Synchronisation every six hours
Azure access	REST client, azure-identity	ARM, Cost Management, Resource Graph, Monitor, Activity Log
Bill import	FOCUS CSV importer	Provider-neutral spend analytics
Secrets	Fernet, scrypt	Encrypted cloud secrets, hashed passwords
Interface	HTML, CSS, JavaScript, SVG	Dashboards, findings, owners, savings, audit
Quality	pytest, GitHub Actions, Docker	27 automated tests, CI, deployment image

VI. Evaluation

We evaluate three things: that the implemented system behaves as specified on real billing data, that the detector and forecaster behave well on controlled series with known answers, and what the evaluation cannot tell us.

A. Functional evaluation on a public bill

The demonstration workspace is built on the FinOps Foundation's anonymised FOCUS sample bill (100,000 line items, licensed CC BY 4.0), aggregated to 1,166 daily service rows, with September 2024 as the reporting month. The reported spend is $2,829.99. The anomaly detector flagged exactly one day, 24 September 2024: spend was $169.26 against a same-weekday median of $73.55, an excess of $95.71 (2.3 times the usual) with a robust score of 7.9 against the threshold of 3.5, and 99.1% of the increase came from one compute service. The next two days stayed high, so the spike was classed as ongoing. Against a budget of $2,600 the forecast crossed the limit on 28 September (109% of budget), the highest-scoring finding (75, band P1). The resource findings in this workspace are scripted example records that run through the same scoring and verification code as a live workspace: 12 findings in total, $262.94 a month still open and $33.44 a month verified, with an estimate accuracy of 94%. These figures show that the pipeline is internally consistent (open and owned stages add up to the headline total); they are not evidence about real savings. The 27 automated tests cover the statistics, scoring, lifecycle, verification, roles, the importer, security checks and the static interface, and all pass.

B. Simulation of the detector

Real billing data with ground-truth anomalies is hard to obtain, so we used a seeded simulation (Python random seeds fixed in the released script). A daily series of 120 days has a base level of 100, a 0.2% daily trend, weekends 22% cheaper, and log-normal noise with σ = 0.08 (quiet) or 0.15 (noisy). One spike of size k times the day's value is injected at a random day from 45 to 116, 400 trials per setting, and a detection counts only if the injected day is flagged. We compare the CloudPulse detector with the textbook rule 'value above the mean plus three standard deviations of the previous 14 days'. False alarms are flagged days per 100 clean days over 300 series.

Table 2 - Detection rate of injected spikes (400 trials per row)

Noise σ	Spike size	CloudPulse	Mean + 3σ rule
0.08	1.25×	60.8%	26.0%
0.08	1.5×	97.0%	71.5%
0.08	2.0×	100.0%	98.5%
0.08	3.0×	100.0%	100.0%
0.15	1.25×	39.0%	14.5%
0.15	1.5×	68.2%	46.3%
0.15	2.0×	96.0%	90.5%
0.15	3.0×	100.0%	100.0%

Table 2 gives the results. CloudPulse detected more injected spikes than the baseline at every size below 3× and equal at 3×, and the gap is largest for moderate spikes: at 1.5 times the usual cost and σ = 0.08 it found 97% against 72%. Both methods found essentially all spikes of three times the usual cost. The price is false alarms: CloudPulse flagged 2.75 and 7.02 days per 100 clean days at σ = 0.08 and 0.15, against 0.16 and 1.12 for the baseline. The same-weekday baseline has only four values, so its estimate of spread is itself noisy; the financial-significance condition (excess of at least 20%) removes the smallest of these, and raising the threshold above 3.5 is the obvious tuning knob for noisy workloads. We report the trade-off rather than a single score because the right operating point depends on how costly a false alarm is to the owner.

C. Simulation of the forecaster

We generated 200 series of 150 days with a 0.4% daily trend, weekend effect and σ = 0.08 noise, and at 2,200 origins forecast the cost seven days ahead from a 60-day history. Holt with weekday factors (the CloudPulse method) was compared with a seven-day moving average and with a naive forecast equal to the last observed day, which for a seven-day horizon is the value on the same weekday of the previous week (Table 3).

Table 3 - Seven-day-ahead forecast error (mean absolute percentage error, 2,200 origins)

Method	MAPE
Holt with weekday factors (CloudPulse)	9.99%
Seven-day moving average	11.51%
Same-weekday naive (last observation)	9.03%

Holt with weekday factors reduced the error of the moving average by 13%, but did not beat the same-weekday naive forecast on this stationary-noise series. That is an honest limit of the method at a

short horizon: its advantage is not accuracy over a one-week lag but that it extrapolates the trend, produces an interval, and projects the whole month for the budget check, none of which the naive forecast does.

D. Threats to validity

The evaluation has clear limits. The bill used for the functional results is a public sample, not our own workload. The resource-level findings in the demonstration are scripted, so the $33.44 verified saving demonstrates the mechanism, not realised savings. The simulation uses a simple generating process; real bills have step changes, monthly commitments and holidays. The 1.5 times, 2.0 times and similar spike sizes are our choices. We have not run a field study with an organisation's production subscription, and we have not compared against commercial tools. A scripted Azure environment that plants real waste exists for such validation and is described in the project documentation, but its results are not part of this paper.

VII. Conclusion and Future Work

CloudPulse shows that a small system built on inspectable methods can carry a cost problem from signal to verified saving while holding only read-only cloud permissions. Its distinguishing choices are accountability (a named creator for each finding), honesty about savings (counted only after the bill confirms them) and safety (a service principal that cannot change the environment). The robust same-weekday detector finds moderate spikes that a mean-and-deviation rule misses, at the cost of more false alarms, and Holt forecasting with weekday factors improves on a moving average while remaining easy to explain.

Future work includes a field study on live subscriptions with real ground truth; tuning of the detector threshold per workload and a comparison with seasonal models; GPU-aware checks, because the present idle verdict uses CPU and network only and could misjudge a machine that trains models on its GPU; categorisation of AI and machine-learning spend with its own budget; resource-level checks for AWS and Google Cloud; and a natural-language layer that explains findings, kept separate from detection so that every flagged item remains explainable.

Declarations

Data and code availability. The bundled sample bill is the FinOps Foundation's public FOCUS sample (CC BY 4.0). The simulation script and the results file used in Section VI are part of the project source, which is available from the authors [add repository URL before publishing]. **Competing interests.** The authors declare no competing interests. **Funding.** This work received no external funding.

References

[1] M. Armbrust et al., "A view of cloud computing," Communications of the ACM, vol. 53, no. 4, pp. 50–58, 2010.

[2] J. R. Storment and M. Fuller, Cloud FinOps: Collaborative, Real-Time Cloud Financial Management, 2nd ed. Sebastopol, CA: O'Reilly Media, 2023.

[3] Cloud Custodian Project, "Cloud Custodian: rules engine for cloud security, cost optimization and governance," https://cloudcustodian.io.

[4] FinOps Foundation, "FinOps Open Cost and Usage Specification (FOCUS)," https://focus.finops.org.

[5] C. Leys, C. Ley, O. Klein, P. Bernard and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," Journal of Experimental Social Psychology, vol. 49, no. 4, pp. 764–766, 2013.

[6] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," Journal of the American Statistical Association, vol. 88, no. 424, pp. 1273–1283, 1993.

[7] B. Iglewicz and D. C. Hoaglin, How to Detect and Handle Outliers. Milwaukee, WI: ASQC Quality Press, 1993.

[8] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," International Journal of Forecasting, vol. 20, no. 1, pp. 5–10, 2004 (reprint of the 1957 Office of Naval Research memorandum).

[9] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne: OTexts, 2021.

[10] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proceedings of the IEEE, vol. 63, no. 9, pp. 1278–1308, 1975.

[11] Microsoft, "What is Azure role-based access control (Azure RBAC)?" Microsoft Learn documentation, https://learn.microsoft.com/azure/role-based-access-control/overview.

[12] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," RFC 6749, Internet Engineering Task Force, 2012.

[13] Fernet specification, https://github.com/fernet/spec.

[14] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," RFC 7914, Internet Engineering Task Force, 2016.